

# Sketches pour la comparaison de génomes

k-mers, MinHash, partitions & HyperMinHash

---

TP JC2BIMMM

Schémas inspirés du billet de C. Marchet

# Comparer deux génomes

On veut mesurer à quel point deux génomes **se ressemblent**.

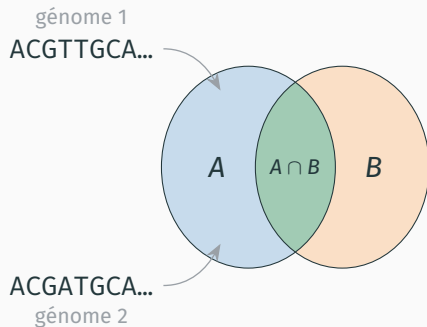
Idée : découper chaque génome en **k-mers** (mots de  $k$  nucléotides), puis comparer les **ensembles** de k-mers.

**Indice de Jaccard :**

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1]$$

Plus  $J$  est grand, plus les génomes partagent de k-mers.

**Problème :** un génome a des *millions* de k-mers  $\Rightarrow$  on veut un **résumé compact** (un *sketch*).

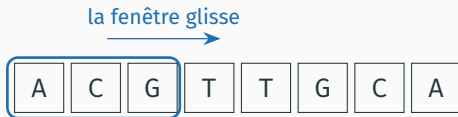


# Énumérer les k-mers

---

# Énumérer les k-mers : par sous-chaînes

Une séquence de longueur  $n$  contient  $n - k + 1$  k-mers (fenêtre glissante). La méthode naïve extrait une sous-chaîne  $\text{seq}[i:i+k]$ .



k-mers :    ACG    CGT    GTT    TTG    TGC    GCA

## Complexité (méthode sous-chaînes)

- Extraire un k-mer (copier  $k$  lettres) :  $O(k)$
- Énumérer *tous* les k-mers :  $O(n \cdot k)$  en temps,  $O(k)$  mémoire/k-mer
- Comparer deux k-mers (chaînes) :  $O(k)$  — lettre par lettre

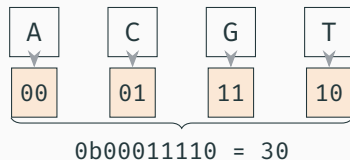
# Encoder un k-mer en entier (2 bits / nucléotide)

Chaque nucléotide tient sur **2 bits** :

A=00 C=01 G=11 T=10

(obtenus par `(ord(c)>>1)&0b11`)

Un k-mer  $\Rightarrow 2k$  bits, gardé dans un seul entier avec un masque `mask = (1<<2k)-1`.



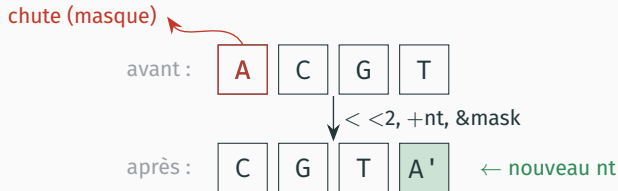
## La limite des 64 bits

Un mot machine fait 64 bits  $\Rightarrow 2k \leq 64 \Rightarrow k \leq 32$ . Comparer deux entiers  $\leq 64$  bits coûte  $O(1)$  (une instruction CPU), au lieu de  $O(k)$  pour des chaînes.

# Énumérer en fenêtre glissante d'entiers

Au lieu de ré-encoder  $k$  lettres à chaque position, on **met à jour** l'entier :

$$\text{kmer} = ((\text{kmer} \ll 2) + \text{val}) \& \text{mask}$$



K-mer **canonique** :  $\min(\text{forward}, \text{reverse-complement}) \Rightarrow$  indépendant du brin.

## Complexité (fenêtre glissante)

$O(1)$  par base  $\Rightarrow O(n)$  en temps,  $O(1)$  mémoire/k-mer.

Gain  $\times k$  (naïf  $O(n \cdot k)$ ).

## **Hachage & Bottom-s MinHash**

---

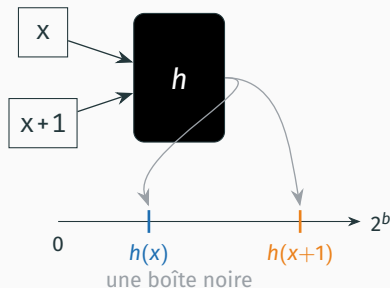
# Qu'est-ce qu'une fonction de hachage ?

Une fonction de hachage  $h$  prend un entier et en renvoie un autre, en **mélangeant ses bits**. Trois propriétés suffisent :

- **déterministe** : même entrée  $\rightarrow$  même sortie ;
- **(quasi-)uniforme** sur  $[0, 2^b)$  ;
- **casse la proximité** : deux k-mers voisins  $\rightarrow$  valeurs très éloignées.

On veut de la *diversité*, pas des k-mers qui se ressemblent.

**Une boîte noire** : peu importe *comment* elle calcule.  
Il en existe beaucoup (MurmurHash, xxHash, xorshift...) — on en choisit une.



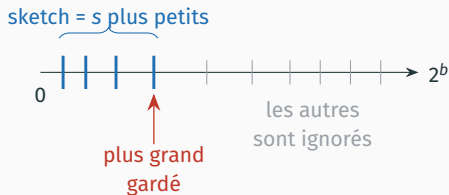
# Bottom-s MinHash : garder les s plus petits hachés

Sélectionner les **s plus petits hachés** = tirer un **échantillon aléatoire uniforme** des k-mers, *cohérent* d'un génome à l'autre (mêmes k-mers → mêmes petits hachés).

**Principe à coder** : maintenir l'ensemble des s plus petits hachés vus.

- moins de s gardés : on **ajoute** ;
- sinon, si le nouveau < **plus grand gardé** : il le **remplace** ;
- sinon : on l'**ignore**.

$$\text{Jaccard estimé : } \hat{J} = \frac{|S_A \cap S_B|}{|S_A \cup S_B|}.$$

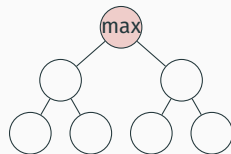


À vous : implémentez `add_kmers`.

## Complexité du bottom-s (+ bonus : tas)

**Version naïve (set) :** recalculer  $\max(\text{kept})$  à chaque remplacement coûte  $O(s) \Rightarrow \mathbf{O(n \cdot s)}$  en temps,  $O(s)$  mémoire.

**Bonus — tas (heapq) :** on stocke -kmer, le max reste à la racine :



tas-max : racine = maximum

- lire le max :  $O(1)$
  - heappushpop (insérer + retirer) :  $O(\log s)$
- $\Rightarrow \mathbf{O(n \log s)}$  en temps.

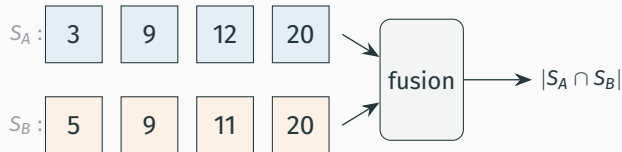
|              | set     | tas           |
|--------------|---------|---------------|
| max          | $O(s)$  | $O(1)$        |
| remplacer    | $O(s)$  | $O(\log s)$   |
| <b>total</b> | $O(ns)$ | $O(n \log s)$ |

## Partition sketch

---

# Le problème : comparer impose un tri

Dans le bottom-s, les  $s$  valeurs gardées dépendent d'un **ordre global** (les plus petites parmi *toutes*). Deux sketches n'ont donc pas les mêmes valeurs aux mêmes positions : pour calculer  $|S_A \cap S_B|$ , il faut **trier / fusionner**.

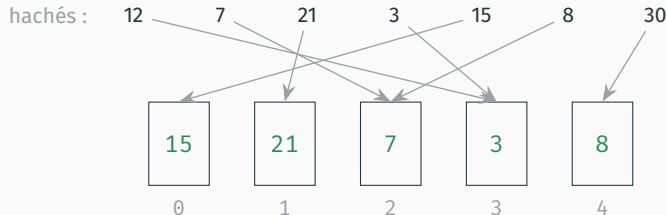


## Coût

Trier / fusionner deux sketches de taille  $s$  :  $O(s \log s)$ .

# Partition sketch de taille $s$

**Idée :** rendre les  $s$  valeurs *indépendantes*. On découpe l'espace en  $s$  **partitions**; chaque k-mer va dans la partition  $p = \text{kmer} \% s$ ; dans chaque partition on ne garde que le **minimum**.



**un minimum par partition**  $\Rightarrow$  cases indépendantes (comparables case à case)

# Complexité du partition sketch

**Construire** : pour chaque k-mer,  $p = \text{kmer} \% s$  ( $O(1)$ ) puis une comparaison  $\Rightarrow O(n)$  au total.

Mémoire :  $s$  minima  $= O(s)$ .

**Comparer** : parcourir les  $s$  slots, une comparaison par slot  $\Rightarrow O(s)$ , sans tri!

$$\hat{j} = \frac{\#\{i : A_i = B_i\}}{\#\text{slots non vides}}$$



4 slots égaux / 5  $\Rightarrow \hat{j} = 0,8$

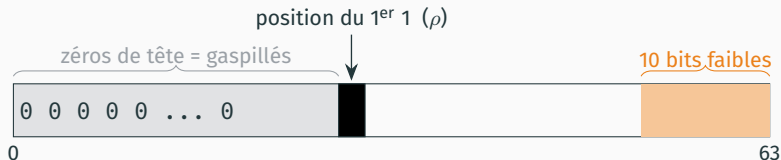
|                    | construire              | comparer                |
|--------------------|-------------------------|-------------------------|
| bottom-s partition | $O(n \log s)$<br>$O(n)$ | $O(s \log s)$<br>$O(s)$ |

**HyperMin : compression 16 bits**

---

# Perte d'espace : les zéros de tête

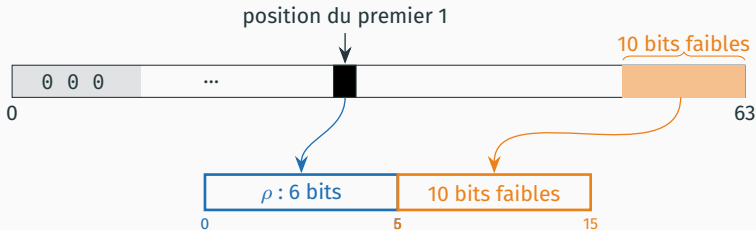
On garde des valeurs *parce qu'elles sont petites* : leur écriture binaire commence donc par **beaucoup de zéros**. L'information utile se résume à **(1)** la position du premier 1 et **(2)** quelques bits de poids faible.



⇒ stocker 64 bits par minimum est un gâchis : on peut résumer l'essentiel sur bien moins de bits.

# HyperMin : un code de 16 bits par partition

Les bits de *poids faible* choisissent la partition ( $p = h \& (s-1)$ ); on encode donc la partie **haute**  $\text{body} = h \gg p\_bits$  sur **16 bits**.



```
def hmh_code(body, w, r=10, rb=6):  
    rho = w if body == 0 else w - body.bit_length()  
    rho = min(rho, (1 << rb) - 1)  
    return (rho << r) | (body & ((1 << r) - 1))
```

$\rho$  = zéros de tête de body.

array('H') : 16 bits.

d'après HyperLogLog, Flajolet et al. 2007

# Espace & compromis mémoire / qualité

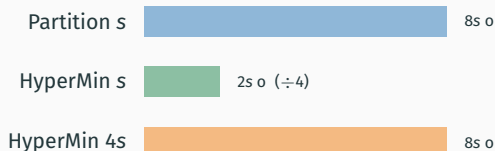
## Mémoire :

- Partition :  $s \times 64 \text{ bits} = 8s \text{ octets}$
- HyperMin :  $s \times 16 \text{ bits} = 2s \text{ octets}$  ( $\div 4$ )

## Deux régimes :

- à **taille égale** : précision quasi identique,  $4\times$  moins de mémoire;
- à **mémoire égale** :  $4\times$  plus de partitions  $\Rightarrow$  écart-type réduit.

**Rançon** : compresser 64  $\rightarrow$  16 bits crée des **collisions**  $\Rightarrow$  légère *surestimation* de  $J$ ;  
négligeable si les partitions sont bien remplies.



à mémoire égale :  $4\times$  plus de partitions  $\rightarrow$  estimation plus stable

## Récapitulatif

---

## Récapitulatif des quatre sketches

| Sketch         | Construire    | Comparer      | Espace       | Tri ? |
|----------------|---------------|---------------|--------------|-------|
| Bottom-s (set) | $O(ns)$       | $O(s \log s)$ | 64s b        | oui   |
| Bottom-s (tas) | $O(n \log s)$ | $O(s \log s)$ | 64s b        | oui   |
| Partition      | $O(n)$        | $O(s)$        | 64s b        | non   |
| HyperMin       | $O(n)$        | $O(s)$        | <b>16s</b> b | non   |

- **Encoder en entiers** : énumération et comparaison passent de  $O(k)$  à  $O(1)$ .
- **Bottom-s** : échantillon uniforme, mais comparaison liée à un ordre global.
- **Partition** : slots indépendants  $\Rightarrow$  comparaison  $O(s)$  sans tri.
- **HyperMin** :  $\div 4$  de mémoire pour une qualité quasi équivalente.

Des k-mers bruts au sketch compact :  
**moins de mémoire, comparaison plus rapide,**  
pour une estimation de Jaccard fiable.